

8 Validation and Form Requests: الدرس

المحتويات

1. Validation مقدمة عن
2. Basic Validation
3. Validation Rules
4. Custom Error Messages
5. Form Request Classes
6. Custom Validation Rules
7. Conditional Validation
8. Array Validation
9. File Validation
10. أمثلة عملية

Validation مقدمة عن

Validation؟ ما هو

Validation = التحقق من صحة البيانات قبل معالجتها أو حفظها في قاعدة البيانات

```
User Input → Validation → ☐ Valid → Process  
→ ☐ Invalid → Show Errors
```

لماذا Validation؟ مهم

حماية قاعدة البيانات من البيانات الخاطئة ^١ تحسين تجربة المستخدم برسائل واضحة ^٢ منع الثغرات الأمنية ^٣ ضمان تناسق ^٤ البيانات

Basic Validation

في Controller

```
use Illuminate\Http\Request;

class PostController extends Controller
{
    public function store(Request $request)
    {
        $validated = $request->validate([
            'title' => 'required|max:255',
            'content' => 'required',
            'email' => 'required|email',
        ]);

        // البيانات صحيحة - يمكن حفظها
        Post::create($validated);

        return redirect()->back()->with('success', 'تم الحفظ بنجاح');
    }
}
```

Blade عرض الأخطاء في

```
@if ($errors->any())
    <div class="alert alert-danger">
        <ul>
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

{{ -- خطأ حقل محدد -- }}
@error('title')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror

{{ -- input في ال -- }}
<input type="text" name="title" value="{{ old('title') }}" class="@error('title')
is-invalid @enderror">
@error('title')
    <span class="invalid-feedback">{{ $message }}</span>
@enderror
```

الحصول على القيم القديمة

```
<input type="text" name="title" value="{{ old('title') }}">
<textarea name="content">{{ old('content') }}</textarea>
<input type="email" name="email" value="{{ old('email', $user->email) }}">
```

Validation Rules

القواعد الأساسية

```
$request->validate([
    // مطلوب
    'name' => 'required',

    // نص
    'title' => 'required|string|max:255',
    'bio' => 'nullable|string|max:500',

    // بريد إلكتروني
    'email' => 'required|email|unique:users,email',

    // رقم
    'age' => 'required|integer|min:18|max:100',
    'price' => 'required|numeric|min:0',

    // تاريخ
    'birth_date' => 'required|date',
    'published_at' => 'nullable|date|after:today',

    // منطقي
    'agree' => 'required|accepted',
    'active' => 'boolean',

    // URL
    'website' => 'nullable|url',

    // Confirmation
    'password' => 'required|string|min:8|confirmed',
    // يجب وجود password_confirmation
]);
```

قواعد النصوص

```
'name' => [
  'required',
  'string',
  'min:3',           // أحرف على الأقل 3
  'max:255',         // حرف كحد أقصى 255
  'alpha',           // حروف فقط
  'alpha_num',       // حروف وأرقام
  'alpha_dash',      // - _ حروف وأرقام و
],

'username' => 'required|string|regex:/^[a-zA-Z0-9_]+$/',
```

قواعد الأرقام

```
'age' => [
  'required',
  'integer',
  'min:18',          // كحد أدنى 18
  'max:100',         // كحد أقصى 100
  'between:18,65',   // بين 18 و 65
],

'price' => 'required|numeric|min:0|max:999999.99',
'quantity' => 'required|integer|digits:4', // أرقام بالضبط 4
```

قواعد التاريخ

```
'birth_date' => [
  'required',
  'date',
  'before:today',    // قبل اليوم
  'after:2000-01-01', // بعد تاريخ محدد
],

'start_date' => 'required|date',
'end_date' => 'required|date|after:start_date',

'published_at' => 'nullable|date|after_or_equal:today',
```

قواعد قاعدة البيانات

```
// Unique - فريد
'email' => 'required|email|unique:users,email',

// Unique (مع استثناء عند التحديث)
'email' => 'required|email|unique:users,email,' . $userId,

أو
'email' => [
    'required',
    'email',
    Rule::unique('users', 'email')->ignore($user->id),
],

// Exists - موجود
'category_id' => 'required|exists:categories,id',
'user_id' => 'required|exists:users,id,active,1', // مع شرط
```

قواعد الملفات

```
'avatar' => [
    'required',
    'image', // صورة فقط
    'mimes:jpeg,png,jpg,gif', // أنواع محددة
    'max:2048', // كحد أقصى 2MB
    'dimensions:min_width=100,min_height=100',
],

'document' => 'required|file|mimes:pdf,doc,docx|max:10240', // 10MB

'video' => 'required|mimetypes:video/mp4,video/avi|max:51200', // 50MB
```

قواعد متقدمة

```
// In - ضمن قائمة
'status' => 'required|in:pending,approved,rejected',
'role' => 'required|in:admin,editor,viewer',

// Not In - ليس ضمن قائمة
'username' => 'required|not_in:admin,root,system',

// Same - مطابق لحقل آخر
'password_confirmation' => 'required|same:password',
```

```
// Different - مختلف عن حقل آخر
'new_password' => 'required|different:old_password',

// Required If - مطلوب إذا
'phone' => 'required_if:contact_method,phone',
'shipping_address' => 'required_if:delivery_type,home',

// Required Unless - مطلوب ما لم
'email' => 'required_unless:contact_method,phone',

// Required With - مطلوب مع
'last_name' => 'required_with:first_name',

// Required Without - مطلوب بدون
'phone' => 'required_without:email',
```

Custom Error Messages

validate() رسائل مخصصة في

```
$request->validate([
    'title' => 'required|max:255',
    'email' => 'required|email',
], [
    'title.required' => 'عنوان المنشور مطلوب',
    'title.max' => 'العنوان يجب ألا يتجاوز 255 حرف',
    'email.required' => 'البريد الإلكتروني مطلوب',
    'email.email' => 'البريد الإلكتروني غير صحيح',
]);
```

أسماء الحقول المخصصة

```
$request->validate([
    'title' => 'required',
    'email' => 'required|email',
], [], [
    'title' => 'عنوان المنشور',
    'email' => 'البريد الإلكتروني',
]);
// الرسالة: "عنوان المنشور مطلوب"
```

رسائل مخصصة في ملف اللغة

resources/lang/ar/validation.php:

```
return [
    'required' => 'مطلوب :attribute حقل',
    'email' => 'يجب أن يكون بريد إلكتروني صحيح :attribute حقل',
    'max' => [
        'string' => 'حرف :max: يجب ألا يتجاوز :attribute حقل',
    ],

    'attributes' => [
        'email' => 'البريد الإلكتروني',
        'title' => 'العنوان',
        'content' => 'المحتوى',
    ],
];
```

Form Request Classes

ما هو Form Request؟

Form Request = يجعل الكود أنظف وأسهل في الصيانة Validation كلاس منفصل للـ.

إنشاء Form Request

```
php artisan make:request StorePostRequest
```

app/Http/Requests/StorePostRequest.php:

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StorePostRequest extends FormRequest
{
    /**
     * تحديد ما إذا كان المستخدم مصرحاً له *
     */
}
```

```

public function authorize(): bool
{
    return true; // أو auth()->check()
}

/**
 * قواعد Validation
 */
public function rules(): array
{
    return [
        'title' => 'required|string|max:255',
        'content' => 'required|string',
        'category_id' => 'required|exists:categories,id',
        'tags' => 'nullable|array',
        'tags.*' => 'exists:tags,id',
        'published_at' => 'nullable|date|after_or_equal:today',
    ];
}

/**
 * رسائل الأخطاء المخصصة
 */
public function messages(): array
{
    return [
        'title.required' => 'عنوان المنشور مطلوب',
        'title.max' => 'العنوان يجب ألا يتجاوز 255 حرف',
        'content.required' => 'محتوى المنشور مطلوب',
        'category_id.required' => 'التصنيف مطلوب',
        'category_id.exists' => 'التصنيف المختار غير موجود',
    ];
}

/**
 * أسماء الحقول المخصصة
 */
public function attributes(): array
{
    return [
        'title' => 'عنوان المنشور',
        'content' => 'محتوى المنشور',
        'category_id' => 'التصنيف',
    ];
}
}

```


Controller الاستخدام في

```
use App\Http\Requests\StorePostRequest;

class PostController extends Controller
{
    public function store(StorePostRequest $request)
    {
        // البيانات صحيحة تلقائياً
        $validated = $request->validated();

        Post::create($validated);

        return redirect()->route('posts.index')
            ->with('success', 'تم إنشاء المنشور بنجاح');
    }
}
```

Authorization في Form Request

```
public function authorize(): bool
{
    // السماح للجميع
    return true;

    // السماح للمستخدمين المسجلين فقط
    return auth()->check();

    // السماح للمستخدم صاحب المنشور فقط
    $post = Post::find($this->route('post'));
    return $post && $this->user()->id === $post->user_id;

    // السماح للمشرفين فقط
    return $this->user()->hasRole('admin');
}
```

Custom Validation Rules

إنشاء Custom Rule

```
php artisan make:rule Uppercase
```

app/Rules/Uppercase.php:

```
<?php

namespace App\Rules;

use Closure;
use Illuminate\Contracts\Validation\ValidationRule;

class Uppercase implements ValidationRule
{
    /**
     * التحقق من القاعدة *
     */
    public function validate(string $attribute, mixed $value, Closure $fail): void
    {
        if (strtoupper($value) !== $value) {
            $fail('يجب أن يكون بأحرف كبيرة :attribute حقل');
        }
    }
}
```

الاستخدام

```
use App\Rules\Uppercase;

$request->validate([
    'code' => ['required', 'string', new Uppercase],
]);
```

Custom Rule مع Parameters

```
<?php

namespace App\Rules;

use Closure;
use Illuminate\Contracts\Validation\ValidationRule;

class MaxWords implements ValidationRule
{

```

```

protected $maxWords;

public function __construct($maxWords)
{
    $this->maxWords = $maxWords;
}

public function validate(string $attribute, mixed $value, Closure $fail): void
{
    $wordCount = str_word_count($value);

    if ($wordCount > $this->maxWords) {
        $fail("كلمة {$this->maxWords} يجب ألا يتجاوز attribute :حقل");
    }
}
}

```

الاستخدام:

```

use App\Rules\MaxWords;

$request->validate([
    'summary' => ['required', 'string', new MaxWords(50)],
]);

```

Closure Rule (قاعدة مباشرة)

```

use Illuminate\Validation\Rule;

$request->validate([
    'email' => [
        'required',
        'email',
        function ($attribute, $value, $fail) {
            if (!str_ends_with($value, '@company.com')) {
                $fail('@company.com يجب أن ينتهي البريد بـ');
            }
        },
    ],
],
]);

```

Conditional Validation

Sometimes Validation

```
$validator = Validator::make($request->all(), [
    'email' => 'required|email',
]);

$validator->sometimes('reason', 'required|max:500', function ($input) {
    return $input->status === 'rejected';
});

if ($validator->fails()) {
    return redirect()->back()->withErrors($validator);
}
```

Conditional Rules في Form Request

```
public function rules(): array
{
    $rules = [
        'title' => 'required|string|max:255',
        'content' => 'required|string',
    ];

    if ($this->input('type') === 'video') {
        $rules['video_url'] = 'required|url';
        $rules['duration'] = 'required|integer';
    }

    if ($this->isMethod('PUT')) {
        $rules['email'] = 'required|email|unique:users,email,' .
        $this->route('user');
    } else {
        $rules['email'] = 'required|email|unique:users,email';
    }

    return $rules;
}
```

Array Validation

Arrays التحقق من

```
$request->validate([
    // Array مطلوب
    'tags' => 'required|array',

    // Array بأدنى وأقصى
    'tags' => 'required|array|min:1|max:5',

    // Array كل عنصر في
    'tags.*' => 'string|max:50',

    // Array من IDs موجودة
    'category_ids' => 'required|array',
    'category_ids.*' => 'exists:categories,id',
]);
```

متداخل Array

```
$request->validate([
    'products' => 'required|array|min:1',
    'products.*.name' => 'required|string|max:255',
    'products.*.price' => 'required|numeric|min:0',
    'products.*.quantity' => 'required|integer|min:1',

    'products.*.images' => 'nullable|array',
    'products.*.images.*' => 'image|max:2048',
]);
```

مثال عملي

```
// الطلب:
[
    'products' => [
        [
            'name' => 'Product 1',
            'price' => 100,
            'quantity' => 5,
            'images' => [/* files */],
        ],
        [
            'name' => 'Product 2',
```

```
        'price' => 200,
        'quantity' => 3,
    ],
],
];

// Validation:
$validated = $request->validate([
    'products' => 'required|array|min:1',
    'products.*.name' => 'required|string|max:255',
    'products.*.price' => 'required|numeric|min:0',
    'products.*.quantity' => 'required|integer|min:1',
    'products.*.images' => 'nullable|array',
    'products.*.images.*' => 'image|mimes:jpeg,png,jpg|max:2048',
]);
```

File Validation

التحقق من الملفات

```
$request->validate([
    // ملف مطلوب
    'document' => 'required|file',

    // صورة
    'avatar' => 'required|image|max:2048', // 2MB

    // أنواع محددة
    'document' => 'required|file|mimes:pdf,doc,docx',

    // حجم محدد
    'video' => 'required|file|max:51200', // 50MB

    // أبعاد الصورة
    'avatar' => [
        'required',
        'image',
        'dimensions:min_width=100,min_height=100,max_width=1000,max_height=1000',
    ],
]);
```

التحقق المتقدم للصور

```
use Illuminate\Validation\Rules\File;

$request->validate([
    'avatar' => [
        'required',
        File::image()
            ->min(100) // 100KB min
            ->max(2048) // 2MB max
            ->dimensions(Rule::dimensions()
                ->minWidth(100)
                ->minHeight(100)
                ->maxWidth(1000)
                ->maxHeight(1000)
            ),
    ],
]);
```

رفع عدة ملفات

```
$request->validate([
    'photos' => 'required|array|min:1|max:5',
    'photos.*' => 'image|mimes:jpeg,png,jpg|max:2048',
]);

// معالجة الملفات
foreach ($request->file('photos') as $photo) {
    $path = $photo->store('photos', 'public');
}
```

أمثلة عملية

مثال 1: نموذج تسجيل مستخدم

StoreUserRequest.php:

```
class StoreUserRequest extends FormRequest
{
    public function authorize(): bool
    {
```

```

        return true;
    }

    public function rules(): array
    {
        return [
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users,email',
            'password' => 'required|string|min:8|confirmed',
            'phone' => 'nullable|string|regex:/^[0-9]{10}$/ ',
            'birth_date' => 'required|date|before:today',
            'avatar' => 'nullable|image|max:2048',
            'agree_terms' => 'required|accepted',
        ];
    }

    public function messages(): array
    {
        return [
            'name.required' => 'الاسم مطلوب',
            'email.required' => 'البريد الإلكتروني مطلوب',
            'email.unique' => 'البريد الإلكتروني مستخدم مسبقاً',
            'password.required' => 'كلمة المرور مطلوبة',
            'password.min' => 'كلمة المرور يجب أن تكون 8 أحرف على الأقل',
            'password.confirmed' => 'تأكيد كلمة المرور غير مطابق',
            'agree_terms.accepted' => 'يجب الموافقة على الشروط',
        ];
    }
}

```

مثال 2: نموذج إنشاء منشور

StorePostRequest.php:

```

class StorePostRequest extends FormRequest
{
    public function authorize(): bool
    {
        return auth()->check();
    }

    public function rules(): array
    {
        return [
            'title' => 'required|string|max:255',
            'slug' => 'required|string|unique:posts,slug|max:255',
        ];
    }
}

```



```

        'content' => 'required|string|min:100',
        'excerpt' => 'nullable|string|max:500',
        'category_id' => 'required|exists:categories,id',
        'tags' => 'nullable|array|max:5',
        'tags.*' => 'exists:tags,id',
        'featured_image' => 'nullable|image|max:2048',
        'status' => 'required|in:draft,published',
        'published_at' =>
            'required_if:status,published|nullable|date|after_or_equal:today',
    ];
}
}

```

مثال 3: نموذج طلب

StoreOrderRequest.php:

```

class StoreOrderRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'items' => 'required|array|min:1',
            'items.*.product_id' => 'required|exists:products,id',
            'items.*.quantity' => 'required|integer|min:1',

            'payment_method' => 'required|in:cash,card,transfer',
            'card_number' => 'required_if:payment_method,card|digits:16',

            'shipping_address' => 'required|string|max:500',
            'shipping_city' => 'required|string|max:100',
            'shipping_zip' => 'required|digits:5',

            'notes' => 'nullable|string|max:1000',
        ];
    }
}

```

نصائح مهمة

أفضل الممارسات ٩

1. Form Requests: استخدم

```
// جيد - منظم وقابل لإعادة الاستخدام ٩
public function store(StorePostRequest $request)
{
    Post::create($request->validated());
}
```

```
// Controller سيء - كود مكرر في ٩
public function store(Request $request) { $validated =
$request->validate([...]); }
```

```
2. فقط استخدم validated() استخدم ٩
```php
// جيد - بيانات صحيحة فقط ٩
$validated = $request->validated();
Post::create($validated);

// خطر - قد تحتوي على بيانات غير صحيحة ٩
Post::create($request->all());
```

### 3. رسائل واضحة:

```
// واضح للمستخدم ٩
'title.required' => 'عنوان المنشور مطلوب'
```

```
// ٩ غير واضح
'title.required' => 'This field is required'
```

```
4. Custom Rules استخدم ٩
```php
// منظم وقابل لإعادة الاستخدام ٩
new Uppercase

// صعب القراءة ٩
'regex:/^[A-Z]+$/'
```

أخطاء شائعة ٩٩

1. في النماذج old() نسيان

```
{{-- ٤ تفقد البيانات عند الخطأ --}}
<input name="title">
```

```
{{-- ٤ تحتفظ بالبيانات --}}
```

```
{{ old('title' )
```

```
2. **استخدام all() بدلاً من validated():**
```php
// ٤ خطر أمني
Post::create($request->all());

// ٤ آمن
Post::create($request->validated());
```

### 3. **authorize()** نسيان:

```
// ٤ أي شخص يمكنه التحديث
public function authorize(): bool
{
 return true;
}
```

```
// ٤ فقط صاحب المنشور public function authorize(): bool { return $this->user()->id ===
$this->route('post')->user_id; }
```

---

## الخطوة التالية

بعد إتمام هذا الدرس، أنت الآن جاهز لـ:

9 الدرس\*\*\*: File Upload and Storage

- File Upload
- Storage Configuration
- File Management

---

\*\*\* ٤! تعلم سعيد \*\*\*